Chapter 3

Making statements

Writing lists

- In Python programming, a variable must be assigned an initial value (initialized) in the statement that declares it in a program, otherwise the interpreter will report a "**not defined**" error.
- A Python "list" is a variable that can store multiple items of data.
- The data is stored sequentially in list "elements" that are index numbered starting at zero.
- So the first value is stored in element zero, the second value is stored in element one, and so on.

- A list is created much like any other variable but is initialized by assigning values as a comma-separated list between square brackets. For example, creating a list named “nums” like this:

  nums = [0 , 1 , 2 , 3 , 4 , 5]
- An individual list element can be referenced using the list name followed by square brackets containing that element’s index number.
- This means that **nums[1]** references the second element in the example above – not the first element, as element numbering starts at zero.

- Lists can have more than one index – to represent multiple dimensions, rather than the single dimension of a regular list.
- Multi-dimensional lists of three indices and more are uncommon but two-dimensional lists are useful to store grid-based information such as X,Y coordinates.

```
#example1
quarter = [ 'January' , 'February' , 'March' ]
print( 'First Month :' , quarter[0] )
print( 'Second Month :' , quarter[1] )
print( 'Third Month :' , quarter[2] )
print( '\nSecond Month First Letter :' , quarter[1][0] )

#example2
coords = [ [ 1 , 2 , 3 ] , [ 4 , 5 , 6 ] ]
print( '\nTop Left 0,0 :' , coords[0][0] )
print( 'Bottom Right 1,2 :' , coords[1][2] )
```

NOTE: String indices may also be negative numbers – to start counting from the right where -1 references the last letter.

Manipulating lists

List Method:	Description:
***list*.append(*x*)**	Adds item *x* to the end of the list
***list*.extend(*L*)**	Adds all items in list *L* to the end of the list
***list*.insert(*i*,*x*)**	Inserts item *x* at index position *i*
***list*.remove(*x*)**	Removes first item *x* from the list
***list*.pop(*i*)**	Removes item at index position *i* and returns it
***list*.index(*x*)**	Returns the index position in the list of first item *x*
***list*.count(*x*)**	Returns the number of times *x* appears in the list
***list*.sort()**	Sort all list items, in place
***list*.reverse()**	Reverse all list items, in place

alpha=A,B,C,D,E

A,B,C,D,E,Z

A,B,X,C,D,E,Z

A,B,X,C,E,Z

B,X,C,E,Z

Z

To manipulate lists

```
basket = [ 'Apple' , 'Bun' , 'Cola' ]
crate = [ 'Egg' , 'Fig' , 'Grape' ]
print( 'Basket List:' , basket )
print( 'Basket Elements:' , len( basket ) )
basket.append( 'Damson' )
print( 'Appended:' , basket )
print( 'Last Item Removed:' , basket.pop() )
print( 'Basket List:' , basket )

basket.extend( crate )
print( 'Extended:' , basket )
del basket[1]
print( 'Item Removed:' , basket )
#len.listname gives you the length
print (len(basket))
del basket[1:3]
print( 'Slice Removed:' , basket )
```

```
>>> list = [0,1,2,3,4,5]
>>> del list[:5]
>>> print(list)
[5]
>>> list = [0,1,2,3,4,5]
>>> del list[1:]
>>> print(list)
[0]
>>> list = [0,1,2,3,4,5]
>>> del list[1:3]
>>> print(list)
[0, 3, 4, 5]
>>> list = [0,1,2,3,4,5]
>>> del list[0]
>>> print(list)
[1, 2, 3, 4, 5]
>>>
```

NOTE: The last index number in the slice denotes at what point to stop removing elements but the element at that position does not get removed.

To manipulate lists

```
>>> list = [0,1,2,3,4,5]
>>> del list[:5]
>>> print(list)
[5]
>>> list = [0,1,2,3,4,5]
>>> del list[1:]
>>> print(list)
[0]
>>> list = [0,1,2,3,4,5]
>>> del list[1:3]
>>> print(list)
[0, 3, 4, 5]
>>> list = [0,1,2,3,4,5]
>>> del list[0]
>>> print(list)
[1, 2, 3, 4, 5]
>>>
```

Restricting lists

- The values in a regular list can be changed as the program proceeds (they are “mutable”) but a list can be created with fixed “immutable” values that cannot be changed by the program.
- A restrictive immutable Python list is known as a “tuple” and is created by assigning values as a comma-separated list between parentheses in a process known as “tuple packing”:

Example:

colors_tuple = (‘Red’ , ‘Green’ , ‘Red’ , ‘Blue’, ‘Red’)

NOTE: Like index numbering with lists, the items in a tuple sequence are numbered from zero.

- An individual tuple element can be referenced using the tuple name followed by square brackets containing that element's index number.
- Usefully, all values stored inside a tuple can be assigned to individual variables in a process known as "sequence unpacking":

```
colors_tuple = ( 'Red' , 'Green' , 'Red' , 'Blue', 'Red' )
a , b , c , d , e = colors_tuple
```

NOTE: There must be the same number of variables as items to unpack a tuple.

Set

- The values in a regular list can be repeated in its elements, as in the tuple above, but a list of unique values can be created where duplication is not allowed.
- A restrictive Python list of unique values is known as a “set” and is created by assigning values as a comma separated list between curly brackets (braces):

```
phonetic_set = { ‘Alpha’ , ‘Bravo’ , ‘Charlie’ }
```

- Individual set elements **cannot** be referenced using the set name followed by square brackets containing an index number, but instead sets have methods that can be dot-suffixed to the set name for manipulation and comparison

Set Method:	Description:
***set*.add(*x*)**	Adds item *x* to the set
***set*.update(*x*,*y*,*z*)**	Adds multiple items to the set
***set*.copy()**	Returns a copy of the set
***set*.pop()**	Removes one random item from the set
***set*.discard(*i*)**	Removes item at position *i* from the set
***set1*.intersection(*set2*)**	Returns items that appear in both sets
***set1*.difference(*set2*)**	Returns items in *set1* but not in *set2*

To demonstrate sets

```
zoo = ( 'Kangaroo' , 'Leopard' , 'Moose' )
print( 'Tuple:' , zoo , '\tLength:' , len( zoo ) )
print( type( zoo ) )

bag = { 'Red' , 'Green' , 'Blue' }
bag.add( 'Yellow' )
print( '\nSet:' , bag , '\tLength' , len( bag ) )
print( type( bag ) )

box = { 'Red' , 'Purple' , 'Yellow' }
print( '\nSet:' , box , '\t\tLength' , len( box ) )
print( 'Common To Both Sets:' , bag.intersection( box ) )
```

Associating list elements

- In Python programming a “dictionary” is a data container that can store multiple items of data as a list of ‘key:value’ pairs. Unlike regular list container values, which are referenced by their index number, values stored in dictionaries are referenced by their associated key.
- The key must be unique within that dictionary and is typically a string name although numbers may be used.

NOTE: In other programming languages a list is often called an “array” and a dictionary is often called an “associative array”.

- Creating a dictionary is simply a matter of assigning the ‘key:value’ pairs as a comma separated list between curly brackets (braces) to a name of your choice.
- Strings must be enclosed within quotes, as usual, and a : colon character must come between the key and its associated value.
- A key:value pair can be deleted from a dictionary by specifying the dictionary name and the pair’s key to the **del** keyword.

- Dictionaries are the final type of data container available in Python programming. In summary, the various types are:
  - Variable – stores a single value
  - List – stores multiple values in an ordered index
  - Tuple – stores multiple fixed values in a sequence
  - Set – stores multiple unique values in an unordered collection
  - Dictionary – stores multiple unordered key:value pairs

```
dc = { 'name' : 'Bob' , 'ref' : 'Python' , 'sys' : 'Win' }
print( 'Dictionary:' , dc )
print( '\nReference:' , dc[ 'ref' ] )

#display all keys within the dictionary
print( '\nKeys:' , dc.keys() )

#delete one pair from the dictionary and add a replacement pair
del dc[ 'name' ]
dc[ 'user' ] = 'Tom'
print( '\nDictionary:' , dc )

#search the dictionary for a specific key
print( '\nIs There A name Key?:' ,'name' in dc )
```

Roll number	Student
1	stu1
2	stu2
3	stu3

1)List all roll numbers

2) Delete the student with roll number 3

3) Check if student with roll number 3 is present

4) add a student with roll number 4

Branching with if

- The Python **if** keyword performs the basic conditional test that evaluates a given expression for a Boolean value of **True or False.**
- This allows a program to proceed in different directions according to the result of the test and is known as “conditional branching”
- The tested expression must be followed by a **: colon,** then statements to be executed when the test succeeds should follow below on separate lines and each line must be indented from the **if** test line.

if statement syntax

if *test-expression :*

statements-to-execute-when-test-expression-is-True

NOTE: Indentation of code is very important in Python as it identifies code blocks to the interpreter – other programming languages use braces.

```
num = int( input( 'Please Enter A Number: ' ) )
if num > 7 and num < 9 :
    print( 'Number is 8' )
if num == 1 or num == 3 :
    print( 'Number Is 1 or 3' )
```

NOTE: The user input is read as a string value by default so must be cast as an **int** data type with **int()** for arithmetical comparison**.**

if-else statement syntax

if *test-expression :*

    statements-to-execute-when-test-expression-is-True

    statements-to-execute-when-test-expression-is-True

else :

    statements-to-execute-when-test-expression-is-False

    statements-to-execute-when-test-expression-is-False

elif statement syntax

if *test-expression-1 :*

    statements-to-execute-when-test-expression-1-is-True

    statements-to-execute-when-test-expression-1-is-True

elif *test-expression-2 :*

    statements-to-execute-when-test-expression-2-is-True

    statements-to-execute-when-test-expression-2-is-True

else :

    statements-to-execute-when-test-expressions-are-False

    statements-to-execute-when-test-expressions-are-False

```
num = int( input( 'Please Enter A Number: ' ) )
if num > 5 :
    print( 'Number Exceeds 5' )
elif num < 5 :
    print( 'Number is Less Than 5' )
else :
    print( 'Number Is 5' )
```

Looping while true

- A loop is a piece of code in a program that automatically repeats.
- One complete execution of all statements within a loop is called an “iteration” or a “pass”.
- The length of the loop is controlled by a conditional test made within the loop.
- While the tested expression is found to be **True** the loop will continue – until the tested expression is found to be **False**, at which point the loop ends.

To demonstrate ‘while’ loop

```
i = 1
while i < 4 :
    print( '\nOuter Loop Iteration:' , i )
    i += 1
    j = 1
    while j < 4 :
        print( '\tInner Loop Iteration:' , j )
        j += 1
```

Looping over items

- The **for** loop in Python is unlike that in other languages such as C as it does not allow step size and end to be specified.

for loop example 1

```
ch = [ 'A' , 'B', 'C' ]
#add statements to display all list element values
print( '\nElements:\t' , end = ' ' )
for item in ch :
    print( item , end = ' ' )
```

for loop example 2

```
for i in range( 1, 4 ) :
    for j in range( 1, 4 ) :
        print( 'Running i=' , i , ' j=' , j )
```

Breaking out of loops

- The Python **break** keyword can be used to prematurely terminate a loop when a specified condition is met.
- The **break** statement is situated inside the loop statement block and is preceded by a test expression.
- When the test returns **True** the loop ends immediately and the program proceeds on to the next task.
- For example, in a nested inner loop it proceeds to the next iteration of the outer loop.

```
for i in range( 1, 4 ) :
    for j in range( 1, 4 ) :
        print( 'Running i=' , i , ' j=' , j )
        if i == 2 and j == 1 :
            print( 'Breaks inner loop at i=2 j=1' )
            break
```

The **continue** statement

- The Python **continue** keyword can be used to skip a single iteration of a loop when a specified condition is met.
- The **continue** statement is situated inside the loop statement block and is preceded by a test expression.
- When the test returns **True** that one iteration ends and the program proceeds to the next iteration.

```
for i in range( 1, 4 ) :
    for j in range( 1, 4 ) :
        if i == 2 and j == 1 :
            print( skipping inner loop at i=2 j=1' )
            continue
        print( 'Running i=' , i , ' j=' , j )
```

Challenges

- Python program that iterates the integers from 1-50, Print the multiples of 3 and 5
- Take a list, and write a program that prints out all the elements of the list that are less than 5.
- Create a program that asks the user for a number and then prints out a list of all the divisors of that number.
- Take 2 Lists as input then write a program that returns a list that contains only the elements that are common between the lists
- Let's say I give you a list saved in a variable: a = [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]. Write a Python program that takes this list a and makes a new list that has only the even elements of this list in it.
- Make a two-player Rock-Paper-Scissors game.
- Generate a random number between 1 and 9 (including 1 and 9). Ask the user to guess the number, then tell them whether they guessed too low, too high, or exactly right.
- Ask the user for a number and determine whether the number is prime or not.